

Objetivos da aula:

- Compreender o conceito e a finalidade das subconsultas em SQL;
- Utilizar subconsultas em diferentes contextos (WHERE, SELECT e FROM);
- Diferenciar subconsultas escalares, de conjunto e correlacionadas;
- Aplicar corretamente os operadores IN, EXISTS, ANY e ALL;
- Interpretar o comportamento das subconsultas em termos lógicos e semânticos.

Índice:

- i. Introdução às Subconsultas
- ii. SQL: EXISTS, ANY, ALL e Subconsultas × JOIN

i. Introdução às Subconsultas**1.1. Contextualização**

Em SQL, nem todas as consultas podem ser expressas de forma simples com um único comando SELECT. Em muitos cenários, é necessário **utilizar o resultado de uma consulta como entrada para outra consulta**. Esse mecanismo é conhecido como **subconsulta** (*subquery*).

As subconsultas permitem expressar **consultas mais complexas**, mantendo clareza semântica e aderência ao modelo relacional. Elas são amplamente utilizadas para:

- Comparar valores com resultados calculados dinamicamente;
- Filtrar registros com base em conjuntos de dados;
- Expressar condições dependentes de outras consultas.

1.2. Conceito de Subconsulta

Uma subconsulta é uma instrução SELECT **aninhada dentro de outra instrução SQL**. A subconsulta é executada primeiro, e seu resultado é utilizado pela consulta principal.

De forma geral:

- A **consulta interna** produz um valor ou conjunto de valores;
- A **consulta externa** utiliza esse resultado em uma condição, expressão ou tabela temporária.

```
DROP TABLE IF EXISTS funcionario;  
CREATE TABLE funcionario (  
    id_funcionario INTEGER PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    salario NUMERIC(10,2)  
);
```

```
INSERT INTO funcionario  
VALUES  
(1, 'Ana', 2000),  
(2, 'Pedro', 1000),  
(3, 'Maria', 3000),  
(4, 'José', 4000),
```

```
(5, 'Carla', 5000);
```

Exemplo conceitual:

“Liste os funcionários cujo salário é maior que a média dos salários.”

Nesse caso:

- A média das notas é obtida por uma subconsulta;

```
SELECT AVG(salario)
FROM funcionario;
```

- O resultado da média é utilizado na consulta principal.

```
SELECT *
FROM funcionario
WHERE salario > (
    SELECT AVG(salario)
    FROM funcionario
);
```

Uma subconsulta é sempre delimitada por parênteses e pode aparecer em diferentes partes da instrução SQL.

Regras importantes:

- A subconsulta **não deve terminar com ponto e vírgula;**

```
SELECT *
FROM funcionario
WHERE salario > (
    SELECT AVG(salario)
    FROM funcionário
);
```

- A subconsulta deve retornar um tipo de dado **compatível** com o operador utilizado. Nesse exemplo, a subconsulta resulta em um número real que é compatível com a comparação `salario > 3000`;
- A consulta externa depende logicamente do resultado da subconsulta.

1.3. Subconsultas no SELECT

O uso mais comum de subconsultas ocorre na cláusula WHERE, porém, subconsultas também podem ser utilizadas na lista de colunas do SELECT, desde que retornem **um único valor**.

```
SELECT nome, salario, salario/(
    SELECT SUM(salario)
    FROM funcionario
) AS porcentagem
FROM funcionario;
```

nome	salario	porcentagem
Ana	2000.00	0.13333333
Pedro	1000.00	0.06666666
Maria	3000.00	0.20000000
José	4000.00	0.26666666
Carla	5000.00	0.33333333

1.4. Subconsulta na Cláusula FROM

Além de serem utilizadas nas cláusulas WHERE e SELECT, as subconsultas também podem aparecer na cláusula FROM. Nesse caso, a subconsulta é tratada como uma **tabela temporária**, também chamada de **tabela derivada**.

Esse recurso é utilizado quando se deseja:

- Executar uma consulta intermediária;
- Utilizar seus resultados como se fossem uma tabela;
- Aplicar novas consultas sobre dados já processados (filtrados, agregados ou calculados).

Uma subconsulta no FROM:

- Deve obrigatoriamente possuir um **alias**;
- Pode conter JOIN, GROUP BY, HAVING, funções de agregação etc.;
- É avaliada antes da consulta externa.

No exemplo a seguir a subconsulta calcula a média no FROM e o valor é usado no SELECT para calcular a diferença entre o salário e a média:

```
SELECT nome, salario, salario - sub.media AS diferenca
FROM funcionario, (
    SELECT AVG(salario) AS media
    FROM funcionario
) AS sub
ORDER BY diferenca DESC;
```

nome character	salario numeric	diferenca numeric
Carla	5000.00	2000.000
José	4000.00	1000.000
Maria	3000.00	0.000
Ana	2000.00	-1000.000
Pedro	1000.00	-2000.000

ii. SQL: EXISTS, ANY, ALL e Subconsultas × JOIN

2.1. Introdução

Neste capítulo, avançaremos para **operadores específicos utilizados com subconsultas**, que ampliam significativamente o poder expressivo da linguagem.

Serão abordados:

- O operador EXISTS;
- Os operadores ANY e ALL;
- A comparação conceitual e prática entre **subconsultas e JOIN**.

2.2. Operador EXISTS

O operador EXISTS é utilizado para **verificar a existência de registros** retornados por uma subconsulta. Ele **não avalia valores**, apenas verifica se a subconsulta retorna **pelo menos uma linha**.

Características do EXISTS

- Retorna TRUE se a subconsulta retornar ao menos um registro;

- Retorna FALSE caso contrário;
- É frequentemente utilizado com **subconsultas correlacionadas**;
- Pode apresentar melhor desempenho do que IN em certos cenários.

Exemplo com EXISTS

“Liste os alunos que possuem ao menos uma matrícula.”

```
SELECT nome
FROM aluno AS a
WHERE EXISTS (
    SELECT *
    FROM matricula AS b
    WHERE a.id_aluno = b.id_aluno
);
```

nome
character varying
Gabriel Souza
Lúcia Garcia
Marcos Silva

Observação didática: O valor **SELECT *** é irrelevante; o que importa é a existência de linhas.

Considere as seguintes tabelas nos exemplos:

```
DROP TABLE IF EXISTS matricula, aluno;
CREATE TABLE aluno (
    id_aluno INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE matricula (
    id_matricula INTEGER PRIMARY KEY,
    id_aluno INTEGER,
    disciplina VARCHAR(100) NOT NULL,
    nota NUMERIC(3,1) NULL,
    FOREIGN KEY (id_aluno)
        REFERENCES aluno(id_aluno)
);
```

```
INSERT INTO aluno (id_aluno, nome)
VALUES
    (1, 'Ana Maria'),
    (2, 'Gabriel Souza'),
    (3, 'Lúcia Garcia'),
    (4, 'Marcos Silva');
```

```
INSERT INTO matricula (id_matricula,
    id_aluno, disciplina, nota)
VALUES
    (1, 2, 'Algoritmos', 7.8),
    (2, 2, 'BD', 6.2),
    (3, 3, 'IoT', 9.1),
    (4, 4, 'Algoritmos', 8.4),
    (5, NULL, 'Álgebra', NULL);
```

2.3. Operador NOT EXISTS

O operador NOT EXISTS verifica a **inexistência de registros** retornados pela subconsulta.

Exemplo com NOT EXISTS

“Liste os alunos que não possuem matrícula.”

```
SELECT nome
FROM aluno AS a
WHERE NOT EXISTS (
    SELECT *
    FROM matricula AS b
    WHERE a.id_aluno = b.id_aluno
);
```

nome
character varying
Ana Maria

2.4. Operadores ANY e ALL

Os operadores ANY e ALL são utilizados quando a subconsulta retorna **um conjunto de valores**, permitindo comparações mais expressivas.

2.4.1 Operador ANY

O operador ANY retorna verdadeiro se a condição for satisfeita para **pelo menos um valor** retornado pela subconsulta.

Exemplo com ANY

*“Liste as disciplinas cuja nota seja maior do que **alguma** das notas da disciplina de Algoritmos.”*

```
SELECT disciplina, nota
FROM matricula
WHERE nota > ANY (
    SELECT nota
    FROM matricula
    WHERE disciplina = 'Algoritmos'
);
```

disciplina character varying (100)	nota numeric (3,1)
IoT	9.1
Algoritmos	8.4

Equivalência conceitual:

- Similar a comparar com o **menor valor** do conjunto, dependendo do operador.

2.4.2 Operador ALL

O operador ALL retorna verdadeiro apenas se a condição for satisfeita para **todos os valores** retornados pela subconsulta.

Exemplo com ALL

*“Liste as disciplinas cuja nota seja maior do que **todas** as notas da disciplina de Algoritmos.”*

```
SELECT disciplina, nota
FROM matricula
WHERE nota > ALL (
    SELECT nota
```

disciplina character varying (100)	nota numeric (3,1)
IoT	9.1

```
FROM matricula
WHERE disciplina = 'Algoritmos'
);
```

Equivalência conceitual:

- Similar a comparar com o **maior valor** do conjunto, dependendo do operador.

2.5. Cuidados com NULL em Subconsultas

É fundamental considerar a presença de valores NULL em subconsultas:

- IN e NOT IN podem produzir resultados inesperados;
- EXISTS e NOT EXISTS são semanticamente mais seguros;
- ANY e ALL também podem ser afetados por NULL, dependendo da comparação.

Boa prática:

Prefira EXISTS quando a lógica envolver apenas a existência de registros.

2.6. Subconsultas × JOIN

Com EXISTS

```
SELECT nome
FROM aluno AS a
WHERE EXISTS (
    SELECT *
    FROM matricula AS b
    WHERE a.id_aluno = b.id_aluno
);
```

Com JOIN

```
SELECT DISTINCT nome
FROM aluno AS a
INNER JOIN matricula AS b
ON a.id_aluno = b.id_aluno;
```

Ambas produzem o mesmo resultado, mas:

- O JOIN combina tabelas;
- O EXISTS testa a existência de relacionamento.

Critérios para Escolha

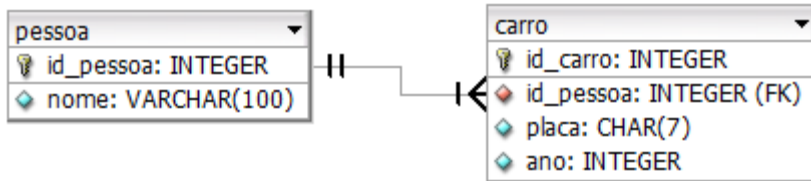
- Use **JOIN** quando:
 - Precisar retornar colunas de múltiplas tabelas;
 - Desejar combinar dados explicitamente;
- Use **subconsultas** quando:
 - A lógica for condicional;
 - A consulta puder ser expressa como “existe / não existe”;
 - Houver necessidade de agregações intermediárias;
- Use **EXISTS** quando:

- A subconsulta depender da linha externa;
- Houver grandes volumes de dados.

Exercícios

Exercício 1 – Subconsulta escalar no WHERE

Elabore uma consulta que liste os carros mais novos que a média.



Resposta:

placa	ano
GHI9C12	2015
TYN5D04	2020

```
DROP TABLE IF EXISTS carro, pessoa;
CREATE TABLE IF NOT EXISTS pessoa (
    id_pessoa INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS carro (
    id_carro INTEGER PRIMARY KEY,
    placa CHAR(7) NOT NULL,
    ano INTEGER,
    id_pessoa INTEGER NOT NULL,
    FOREIGN KEY (id_pessoa)
        REFERENCES pessoa(id_pessoa)
        ON DELETE CASCADE
);
```

```
INSERT INTO pessoa (id_pessoa, nome)
VALUES
    (1, 'Carlos Silva'),
    (2, 'Ana Souza'),
    (3, 'Maria Clara'),
    (4, 'João Bosco');
```

```
INSERT INTO carro
(id_carro, placa, ano, id_pessoa)
VALUES
    (1, 'ABC1A34', 2000, 1),
    (2, 'DEF5B78', 2010, 2),
    (3, 'GHI9C12', 2015, 1),
    (4, 'TYN5D04', 2020, 4);
```

Resposta:

```
SELECT placa, ano
FROM carro
WHERE ano > (
    SELECT AVG(ano)
    FROM carro
);
```

Exercício 2 – Subconsulta escalar no WHERE e JOIN

Inclua na consulta do exercício anterior o nome do proprietário.

Resposta:

```
SELECT a.placa, a.ano, b.nome
FROM carro AS a
INNER JOIN pessoa as b
ON a.id_pessoa = b.id_pessoa
WHERE a.ano > (
    SELECT AVG(ano)
    FROM carro
);
```

Resposta:

placa character (7)	ano integer	nome character varying
GHI9C12	2015	Carlos Silva
TYN5D04	2020	João Bosco

Exercício 3 – Subconsulta com IN

Elabore uma consulta que liste as pessoas que possuem ao menos um carro.

Resposta:

```
SELECT *
FROM pessoa
WHERE id_pessoa IN (
    SELECT id_pessoa
    FROM carro
);
```

Resposta:

id_pessoa [PK] integer	nome character varying (100)
1	Carlos Silva
2	Ana Souza
4	João Bosco

Exercício 4 – Subconsulta com NOT IN

Elabore uma consulta que liste as pessoas que não possuem carro.

Resposta:

```
SELECT *
FROM pessoa
WHERE id_pessoa NOT IN (
    SELECT id_pessoa
    FROM carro
);
```

Resposta:

id_pessoa [PK] integer	nome character varying
3	Maria Clara


```
);
```

Exercício 5 – Subconsulta no WHERE

Elabore uma consulta que liste as pessoas que possuem carro mais novo que a Ana Souza.

Resolva a questão sem usar os operadores ANY ou ALL.

Resposta:

```
SELECT DISTINCT p.nome
FROM pessoa p
JOIN carro c
    ON c.id_pessoa = p.id_pessoa
WHERE c.ano > (
    SELECT MAX(c2.ano)
    FROM pessoa p2
    JOIN carro c2
        ON c2.id_pessoa = p2.id_pessoa
    WHERE p2.nome = 'Ana Souza'
);
```

Resposta:

nome
character varying
Carlos Silva
João Bosco

Exercício 6 – Operadores ANY e ALL

Resolva o exercício anterior usando o operador ANY ou ALL.

Resposta:

```
SELECT DISTINCT p.nome
FROM pessoa p
JOIN carro c
    ON c.id_pessoa = p.id_pessoa
WHERE c.ano > ANY (
    SELECT c2.ano
    FROM pessoa p2
    JOIN carro c2
        ON c2.id_pessoa = p2.id_pessoa
    WHERE p2.nome = 'Ana Souza'
);
```

Resposta:

nome
character varying
Carlos Silva
João Bosco